

RankIt

Community Ranked-Voting Application

By Team: Rankers

Group Project Proposal

Spring 2025

Team Members

Ali Abdulrazzak — Interactive Software Design

Jordan Beirne — Connectivity

Ethan Witkowski — UI/Design

Eddy Umoh — Software Design

Ethan Witkowski — Testing & Presentation/Demo

1. Executive Summary

RankIt is a native Android application that empowers communities to create, discover, and vote on ranked polls across a wide range of topics. Users can browse polls by category — including Sports, Music, Movies, TV Shows, Food, and Gaming — upvote or downvote each option, and watch the rankings shift in real time. The app is designed to be simple, engaging, and community-driven, with safeguards against duplicate content and tools for users to contribute new options to existing polls.

The project will be built using Kotlin and Jetpack Compose for the frontend, with a MongoDB Atlas backend handling all data persistence. The team of five will divide responsibilities across design, development, connectivity, and quality assurance, delivering a functional MVP within three development sprints.

2. App Overview

2.1 Concept

RankIt is a ranked-voting platform where users can express their opinions through upvotes and downvotes on curated lists. For example, in a poll titled "The 12 Greek Olympians," each of the twelve Olympians is listed as a votable option. The one with the most net upvotes rises to the top; the one with the most downvotes sinks to the bottom. This creates a dynamic, community-driven ranking that evolves with participation.

2.2 Core User Stories

- As a user, I want to browse polls by category so I can find topics that interest me.
- As a user, I want to upvote or downvote options so I can influence the ranking.
- As a user, I want to create a new poll with a title, category, and list of options.
- As a user, I want to add a new option to an existing poll so the list stays comprehensive.
- As a user, I want to see live rankings update as votes come in.
- As a user, I want duplicate options to be blocked so polls stay clean and accurate.

2.3 Target Audience

RankIt is designed for casual users aged 16–35 who enjoy ranking and debating topics with friends or strangers online. It suits anyone who has an opinion — sports fans ranking the greatest athletes, music lovers debating the best albums, or film buffs sorting their favorite directors.

3. Team Roles & Responsibilities

The following table outlines each team member's primary role and areas of ownership for this project:

Name	Role	Responsibilities	Contact
Ali Abdulrazzak	Interactive Software Design	Interaction flows, UX logic	
Jordan Beirne	Connectivity	MongoDB integration, API	
Eddy Umoh	Software Design	Architecture, core features	
Ethan Witkowski	Testing & Demo	QA, presentation, demo prep	

Roles may overlap during development; all team members are expected to contribute to code reviews, testing feedback, and the final presentation.

4. Feature Specifications

The following features represent the full scope of the RankIt MVP. Features are prioritized across three development sprints:

Feature	Description	Owner	Sprint
Poll Creation	Users tap the + button to create new ranked-voting polls	Ali	Sprint 1
Upvote / Downvote	Options can be upvoted or downvoted; list reorders live	Ali / Eddy	Sprint 1
Category Sections	Polls organized by Sports, Music, Movies, Food, Gaming, etc.	Ethan	Sprint 1
Image Upload	Users can attach images to voting options and categories	Ali / Eddy	Sprint 2
Duplicate Prevention	System detects and blocks duplicate poll titles and options	Eddy	Sprint 2
Add Options	Community members can suggest new options to existing polls	Ali	Sprint 2
MongoDB Backend	All polls and votes stored and retrieved from MongoDB Atlas	Jordan	Sprint 2
Search & Filter	Full-text search across poll titles and category filtering	Eddy	Sprint 3
User Profiles	Basic user accounts with vote history and created polls	Ethan / Jordan	Sprint 3

Testing & QA	Unit tests, integration tests, and final demo preparation	Ethan	Sprint 3
--------------	---	-------	----------

4.1 Poll Creation (+ Button)

A floating action button (FAB) with a + icon is always visible on the home screen. Tapping it opens a creation flow where users enter a poll title, select a category, and add at least two options. The system validates for duplicate titles before saving.

4.2 Upvote & Downvote

Every poll option displays an upvote arrow and a downvote arrow. Tapping upvote increments the option's score by one; tapping downvote decrements it (minimum 0). The list reorders in real time based on the new scores, with animated transitions.

4.3 Image Upload

When creating a poll or adding an option, users can optionally attach an image. Images are stored via the backend and displayed as thumbnails alongside the option name in the ranked list.

4.4 Duplicate Prevention

The backend enforces uniqueness at two levels: poll titles (case-insensitive, per category) and option names within a single poll. If a user attempts to submit a duplicate, a clear error message is shown and the submission is rejected.

4.5 Category Sections

The home screen features horizontally scrollable category filter chips. Selecting a category filters the poll feed to show only polls tagged with that category. An "All" option shows the full unfiltered list.

5. Technical Architecture

5.1 Technology Stack

Layer	Technology	Purpose
Frontend	Kotlin + Jetpack Compose	Native Android UI toolkit
Backend	MongoDB Atlas	Cloud-hosted NoSQL database
Networking	Retrofit + OkHttp	REST API client for Android
Architecture	MVVM + StateFlow	ViewModel-driven reactive state
Image Loading	Coil	Async image loading for Compose

Navigation	Jetpack Navigation Compose	In-app screen routing
Version Control	Git / GitHub	Source control and collaboration
Build System	Gradle 8.2	Android project build tooling

5.2 Architecture Pattern

The app follows the MVVM (Model-View-ViewModel) architecture pattern recommended by Google for Android development. The View layer is composed entirely of Jetpack Compose composables. ViewModels hold and expose UI state via Kotlin StateFlow. The Repository layer abstracts data access, interfacing with MongoDB Atlas through a Retrofit-based REST API.

5.3 Data Model

- Poll: { id, title, category, author, emoji, options[], createdAt }
- PollOption: { id, pollId, name, votes, imageUrl }
- User: { id, username, createdPolls[], voteHistory[] }

5.4 MongoDB Integration

Jordan Beirne will implement the connectivity layer using MongoDB Atlas as the cloud database. The Android app will communicate with Atlas through a REST API (Retrofit), with endpoints for fetching polls, submitting votes, creating polls, and adding options. Duplicate detection logic will be enforced server-side.

6. Project Timeline

Sprint	Duration	Deliverables	Owner(s)
Sprint 1	Weeks 1–2	Project setup, navigation, home screen, poll cards, category chips, upvote/downvote UI	All
Sprint 2	Weeks 3–4	MongoDB backend, poll creation, duplicate detection, add option, image upload	Jordan, Eddy, Ali
Sprint 3	Weeks 5–6	Search, user profiles, polish, testing, bug fixes, demo preparation	Ethan
Demo Day	Week 7	Final presentation, live demo, code freeze	Ethan

7. Risks & Mitigations

- MongoDB connectivity delays — Mitigated by building the app with an in-memory repository first; swap in the real DB once the API is ready.
- Scope creep — Feature set is locked at the requirements above; additional features are out of scope for the MVP.
- Merge conflicts — Weekly code reviews and feature branches on GitHub will keep the main branch stable.
- Team availability — Responsibilities are clearly assigned; if a member is unavailable, their tasks are documented and can be picked up by another.
- Image upload complexity — Image upload is a Sprint 2 feature with fallback to emoji-only if it cannot be completed in time.

8. Success Criteria

The project will be considered successful if the following criteria are met by Demo Day:

- Users can create a poll with a title, category, and at least two options.
- Upvoting and downvoting options reorders the ranked list correctly.
- The application prevents duplicate poll titles and duplicate options within a poll.
- Polls are persisted to and retrieved from MongoDB Atlas.
- The app runs without crashes on an Android emulator (API 26+).
- At least four poll categories are functional and filterable.
- A live demo can be performed in under 10 minutes.

9. Conclusion

RankIt represents a practical, community-oriented application that exercises the full Android development stack from Compose UI to cloud-backed persistence. With clearly defined roles, a phased sprint plan, and a focused feature set, the team is well-positioned to deliver a polished MVP by Demo Day. We are excited to build something genuinely fun and useful — and we believe ranked community voting is a compelling mechanic that keeps users engaged and coming back.

We look forward to feedback on this proposal and are prepared to begin Sprint 1 immediately upon approval.

— RankIt Team, Spring 2025 —